Inteligência Computacional: Introdução à classificação com “nearest neighbor”

Aldebaro Klautau

Federal University of Para (UFPA)

2026

Classification

Problem:

- Given a vector **x** with *N* parameters, find its class *y* among *Y* possible labels

vector **x** → classifier → *y* or

Fish classifier:

or ?

Training set
used during
the training stage

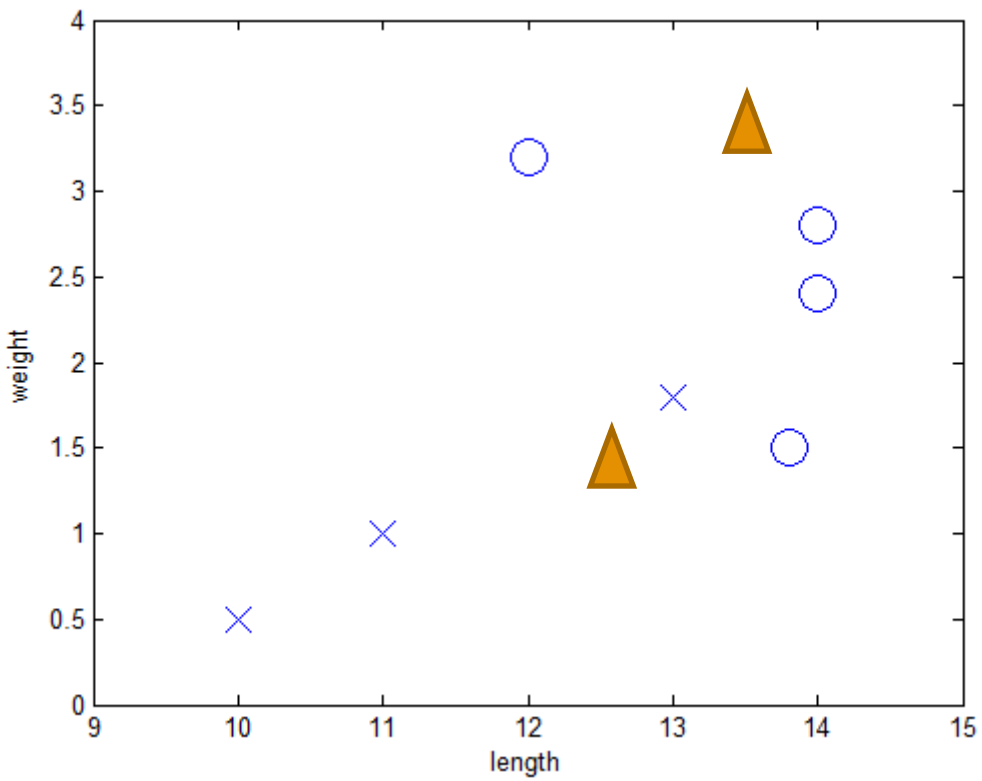
Length	Weight	Class *y*
12	3.2	
10	0.5	
14	2.8	

Test stage uses "unseen" examples:
x = (13, 4.2)
y = ?

Evaluation: misclassification (or error) rate

Nearest neighbor classifier

- The nearest neighbor (NN) classifier simply stores the whole training sequence and, according to the adopted distance measure (e.g. Euclidean distance) assigns to the test example the same class of its nearest (smallest distance) neighbor (example of the stored training sequence
- The Euclidean distance corresponds to the squared value of the error vector norm
  - y represents the test vector
  - z represents a training example
  - Euclidean: distance(y, z) = $|| y - z ||^2$



NN classifier implementation

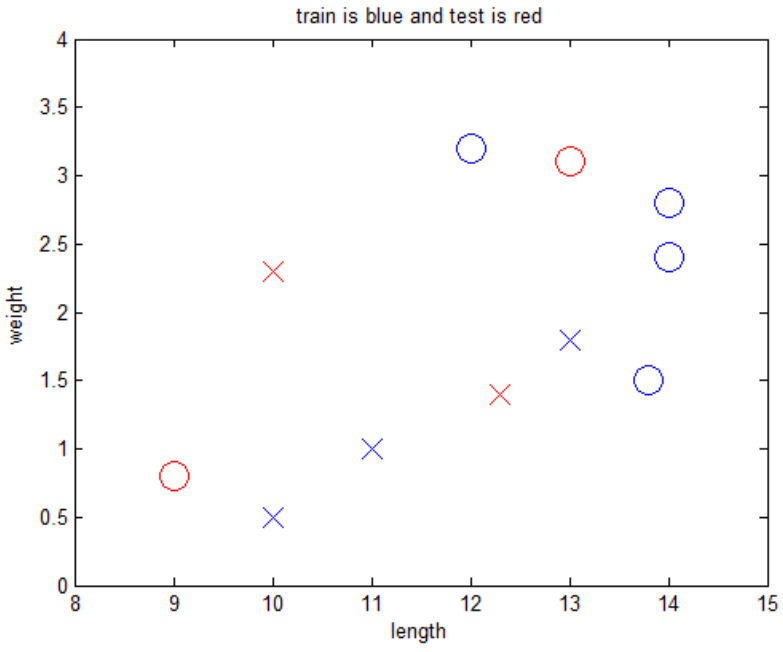
```
x_train=load('train_set.txt'); %load from file to memory
x_test=load('test_set.txt'); %load from file to memory
[N_train,M]=size(x_train); %N_train is the number of examples and M=3
[N_test,M]=size(x_test); %N_test is the number of test examples
numberOfErrors=0; %count number of errors
for i=1:N_test %go over all test set
    testFeatures = x_test(i,1:2);%extract only features (discard label)
    %find nearest neighbor in training set
    minimumDistance = Inf; %initialize with large number
    nearestNeighborIndex = -1;
    for j=1:N_train %go over all training set
        trainFeatures = x_train(j,1:2);%extract only features (discard la
        distance = sum( (testFeatures-trainFeatures).^2 ); %squared norm
        if distance < minimumDistance %check if better than current
```

NN classifier implementation

```
        trainFeatures = x_train(j,1:2);%extract only features (discard la
        distance = sum( (testFeatures-trainFeatures).^2 ); %squared norm
        if distance < minimumDistance %check if better than current
            nearestNeighborIndex = j; %store the current NN
            minimumDistance = distance; %update minimum distance
        end
    end
    classifierOutput = x_train(nearestNeighborIndex,3); %decision
    if classifierOutput ~= x_test(i,3) %compare with correct label
        numberOfErrors = numberOfErrors + 1; %update number of errors
    end
end
```

NN classifier implementation

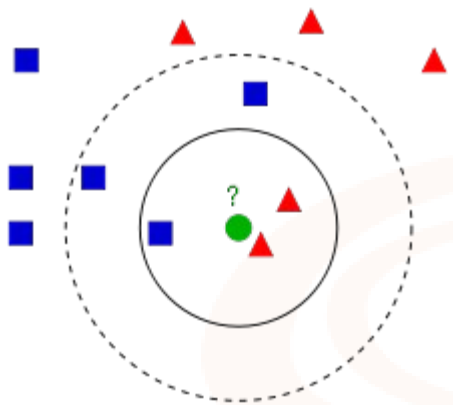
```
x_train=load('train_set.txt'); %load from file to memory
x_test=load('test_set.txt'); %load from file to memory
[N_train,M]=size(x_train); %N_train is the number of examples and M=3
[N_test,M]=size(x_test); %N_test is the number of test examples
numberOfErrors=0; %count number of errors
for i=1:N_test %go over all test set
    testFeatures = x_test(i,1:2);%extract only features (discard labe
    %find nearest neighbor in training set
    minimumDistance = Inf; %initialize with large number
    nearestNeighborIndex = -1;
    for j=1:N_train %go over all training set
        trainFeatures = x_train(j,1:2);%extract only features (discar
        distance = sum( (testFeatures-trainFeatures).^2 ); %squared n
        if distance < minimumDistance %check if better than current
            nearestNeighborIndex = j; %store the current NN
            minimumDistance = distance; %update minimum distance
        end
    end
    classifierOutput = x_train(nearestNeighborIndex,3); %decision
    if classifierOutput ~= x_test(i,3) %compare with correct label
        numberOfErrors = numberOfErrors + 1; %update number of errors
    end
end
```



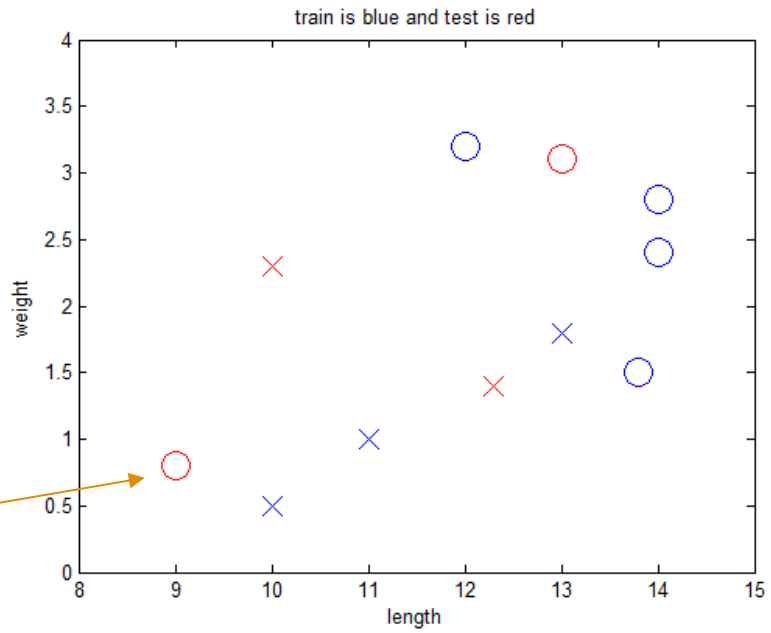
Error rate on test set = 25 %
(error on training set is always 0)

Generalizing the NN: K-nearest neighbors

- KNN classifier: choose K as an integer odd number (e.g. K=3 or 5 is widely adopted) and make the classifier to output the most popular label among the K nearest neighbor as final decision
- The previous NN classifier is equivalent to using K=1, which may be less robust to *outliers* than K > 1
- Example: assume that all red and blue examples compose a new training sequence. Note the outlier!

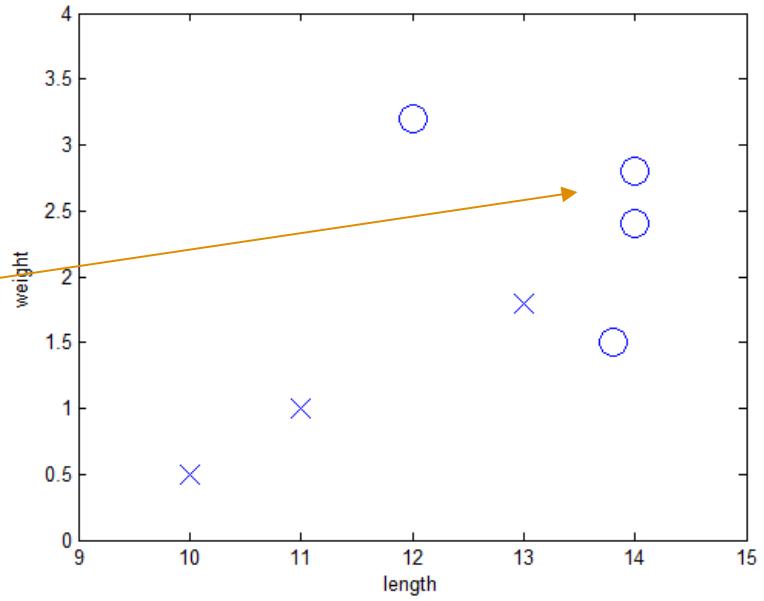


Example from Wikipedia
en.wikipedia.org/wiki/K-nearest_neighbors_algorithm



Controling number of examples in KNN

- KNN is called "lazy" classifier because "training" corresponds to storing the whole training sequence. But this can lead to a prohibitive computational cost
- To decrease the number of examples stored per class, a "clustering" algorithm such as K-means can be used (the meaning for K in K-means is **not** same as K in K-NN)
  - Example, running K-means with K=2 for the 4 examples of label 0 would return 2 new vectors to represent the class 0
- K-means is an iterative algorithm, but in the special case of K=1 with Euclidean distance, its outputs is simply the average of the input vectors
  - Running a single K-means with K=1 "mean" per class, would lead to one vector representing each class



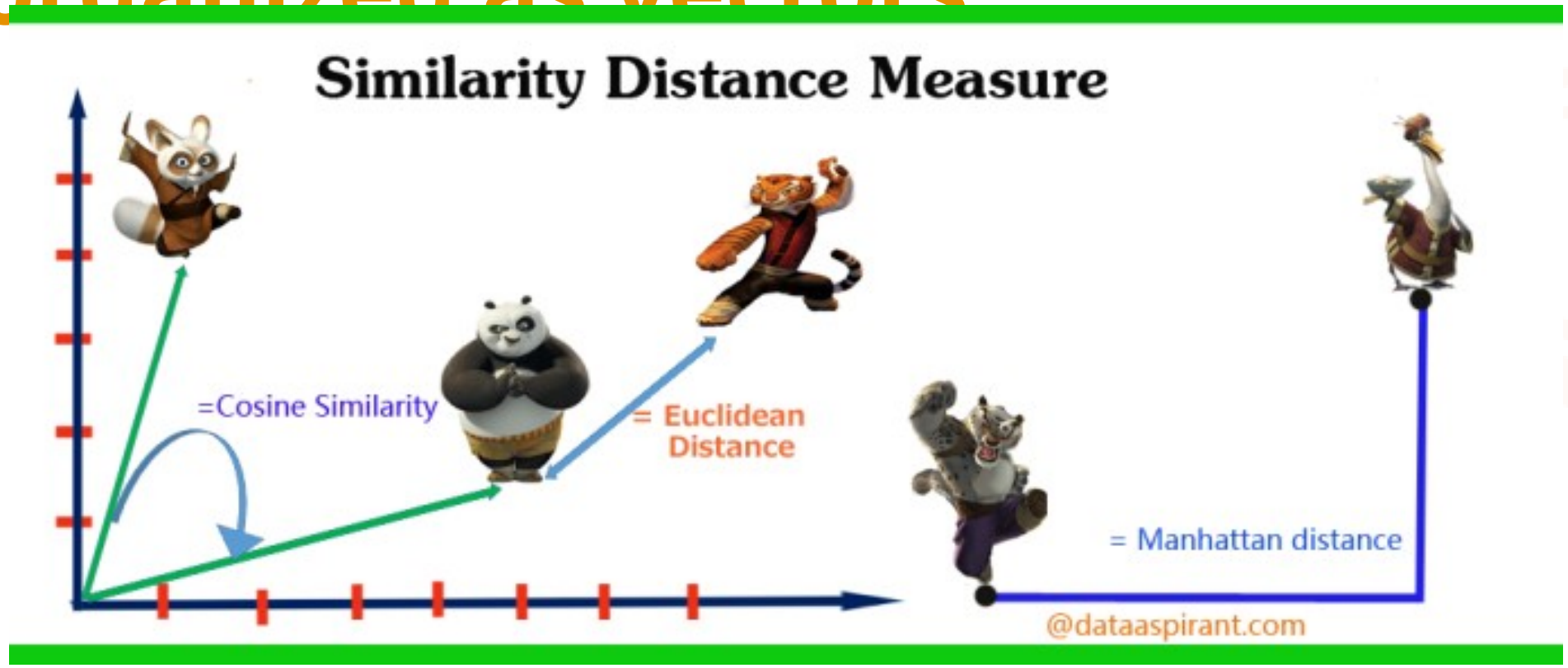
Norms

- Besides the Euclidean distance, there are several norms that can be used to compare (error) vectors and matrices. Norms are used to define distances:
  d(x,y) = || x-y ||

	Vector	Matrix
one-norm	$\|x\|_1 = \sum_i \|x_i\|$	$\|A\|_1 = \max_j \sum_i \|a_{ij}\|$
two-norm	$\|x\|_2 = (\sum_i \|x_i\|^2)^{1/2}$	$\|A\|_2 = \max_{x \neq 0} \|Ax\|_2 / \|x\|_2$
Frobenius norm	$\|x\|_F = \|x\|_2$	$\|A\|_F = (\sum_{ij} \|a_{ij}\|^2)^{1/2}$
infinity-norm	$\|x\|_\infty = \max_i \|x_i\|$	$\|A\|_\infty = \max_i \sum_j \|a_{ij}\|$

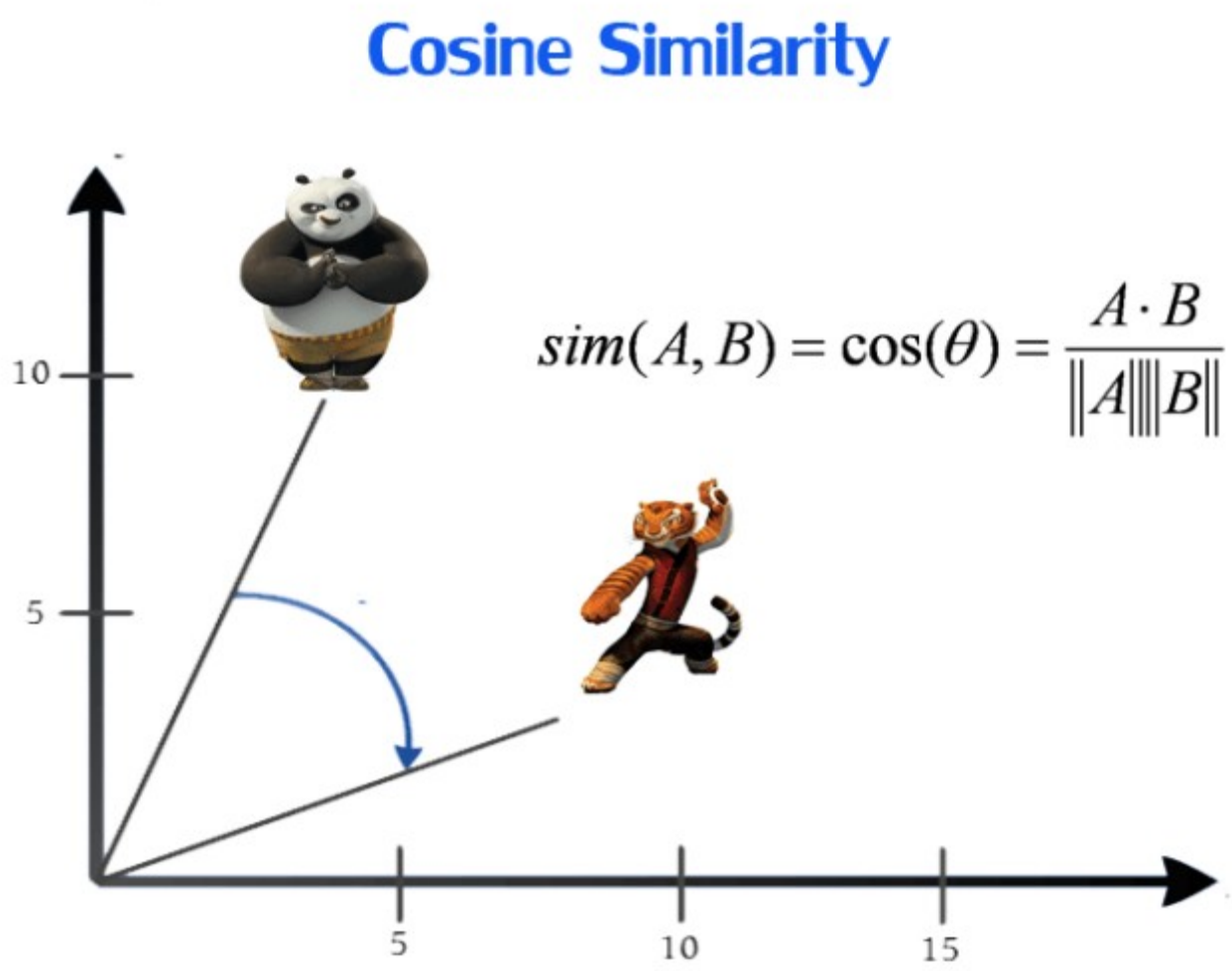
- norm([3 4],1)=7
- norm([3 4]) = 5
- norm([3 4],'fro')=5
- norm([3 4],Inf) = 4
- http://www.netlib.org/lapack/lug/node75.html

There are several similarity measures for features organized as vectors



http://dataaspirant.com/2015/04/11/five-most-popular-similarity-measures-implementation-in-python/

Similarity based on inner product

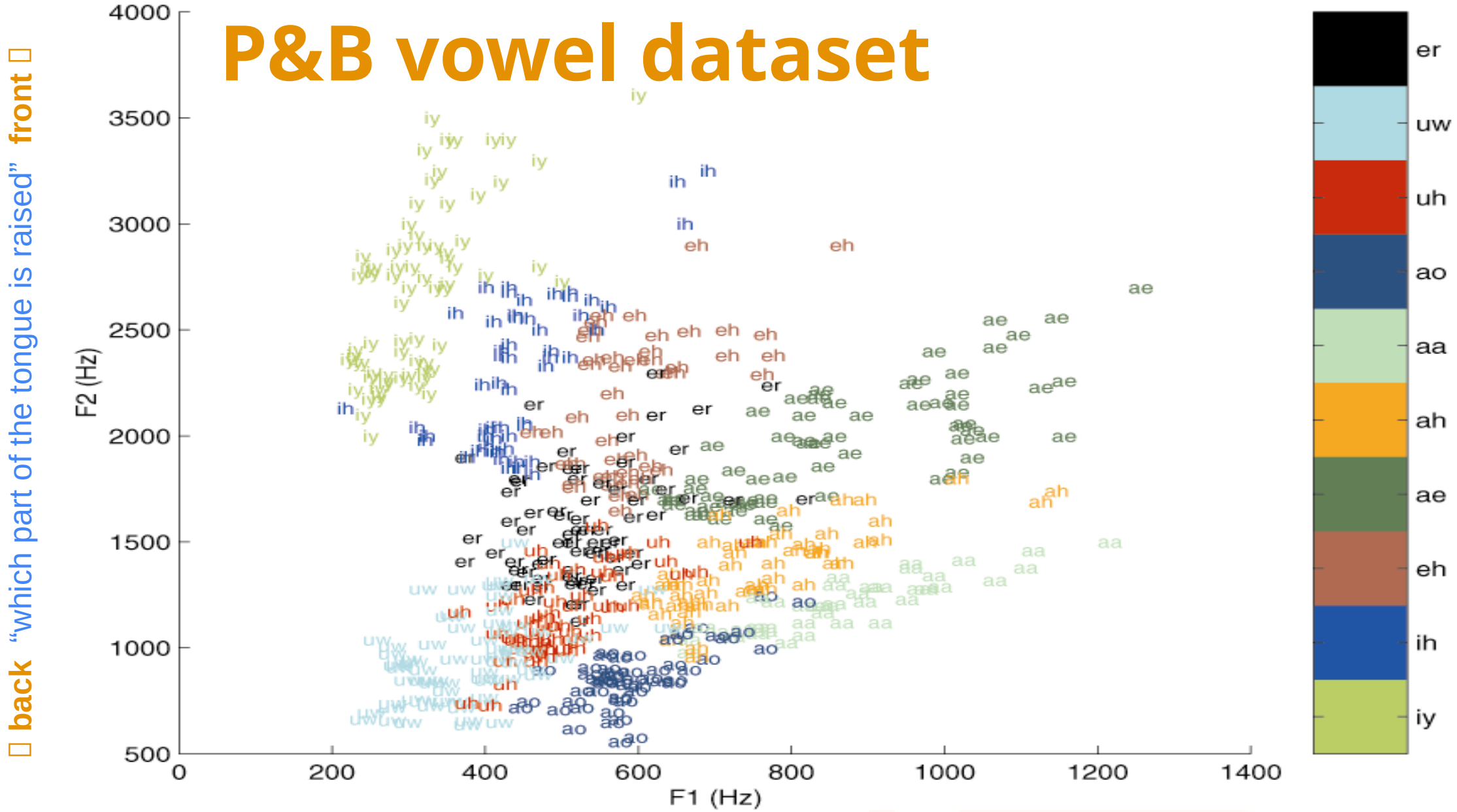
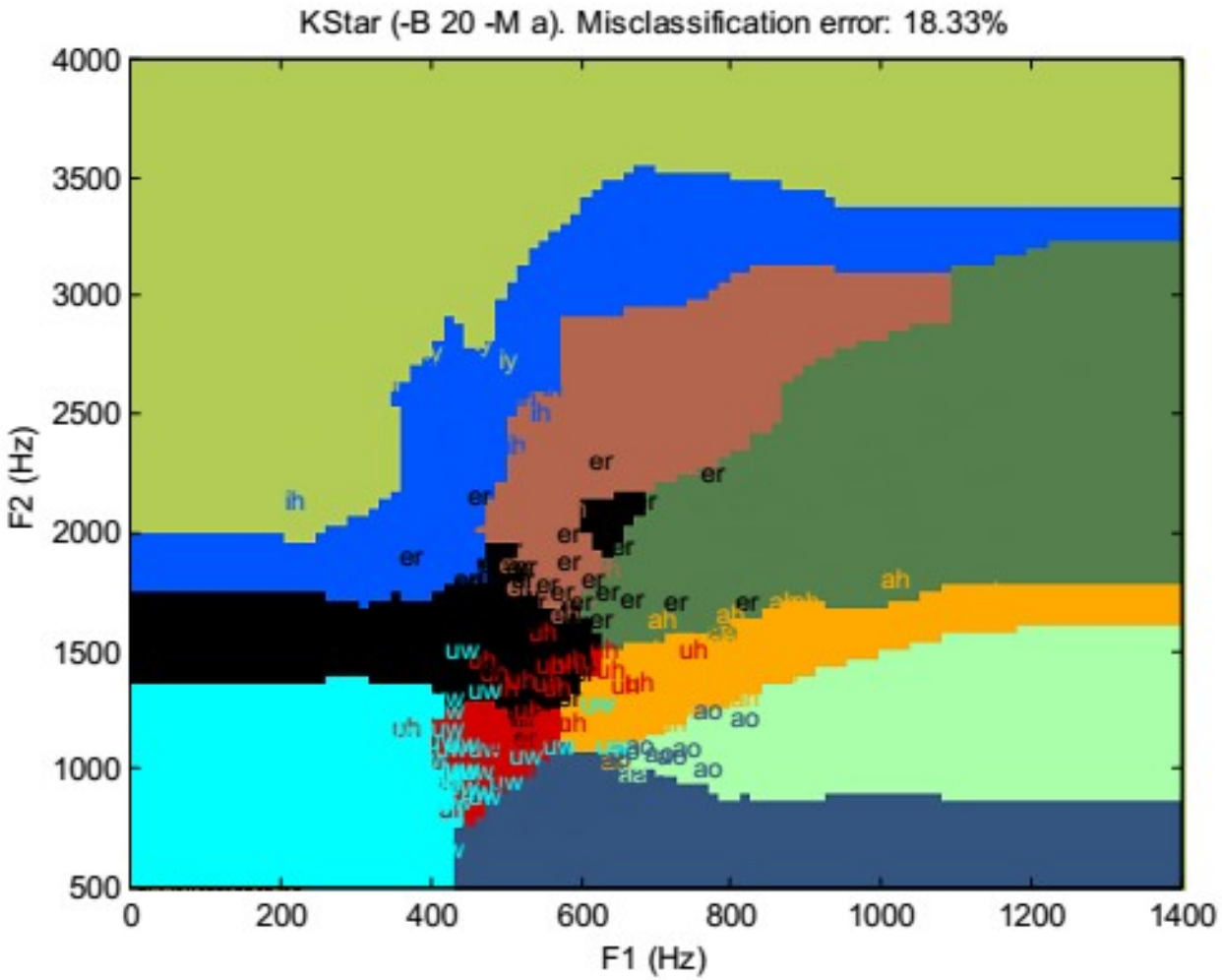


http://dataaspirant.com/2015/04/11/five-most-popular-similarity-measures-implementation-in-python/

Decision regions

Examples with different classifiers using Peterson & Barney's vowel dataset

P&B vowel dataset



Decision region for “KStar” classifier